Quick Sort — Exam Notes

1. Theory

Quick Sort is a **Divide and Conquer** sorting algorithm. It works by selecting a **pivot** and partitioning the array around the pivot.

Three main steps

1. **Pivot Selection:** Select an element as the pivot.
2. **Partitioning:** Arrange the array so that:
    - Elements **smaller than or equal to pivot** are on the left.
    - Elements **greater than pivot** are on the right.
3. **Recursion:** Apply Quick Sort recursively to the left and right subarrays.

After partitioning, the pivot reaches its **correct sorted position**.

---

2. Example

Given:

[
[9,-3,5,2,6,8,-6,1,3]
]

Here, **last element is selected as pivot**.

[
pivot=3
]

Partition:

9 -3 5 2 6 8 -6 1 3
↑
pivot

After partitioning:

[
[-3,\ 2,\ -6,\ 1,\ 3,\ 9,\ 5,\ 6,\ 8]
]

So:

[-3, 2, -6, 1] [3] [9, 5, 6, 8]
    LEFT   PIVOT   RIGHT

Notice that the left and right parts are **not necessarily sorted yet**.

Then Quick Sort is recursively applied:

[-3, 2, -6, 1] 3 [9, 5, 6, 8]
    ↓   ↓
  Quick Sort   Quick Sort

Eventually:

[
\boxed{[-6,-3,1,2,3,5,6,8,9]}
]

---

3. Partition Algorithm

Algorithm: Partition

Step 1: Start with `low` and `high`.

Step 2: Select the last element as pivot:
`pivot = arr[high]`

Step 3: Initialize:

- `i = low`
- `j = low`

Step 4: Repeat while `i <= high`:

- If `arr[i] <= pivot`:
    - Swap `arr[i]` and `arr[j]`

        - Increment `j`
- Increment `i`

Step 5: Return `j - 1` as the partition index.

Sample Code — Java

```
public class QuickSort {

    static int partition(int[] arr, int low, int high) {

        int pivot = arr[high];
        int i = low;
        int j = low;

        while (i <= high) {

            if (arr[i] <= pivot) {

                int temp = arr[j];
                arr[j] = arr[i];
                arr[i] = temp;

                j++;
            }

            i++;
        }

        return j - 1;
    }

    public static void main(String[] args) {

        int[] arr = {10, 7, 8, 9, 1, 5};

        int low = 0;
        int high = arr.length - 1;

        int p = partition(arr, low, high);

        System.out.println("Partition Index: " + p);

        for (int x : arr) {
            System.out.print(x + " ");
        }
    }
}
```

6. Complexity

Case	Time Complexity
Best Case	**O(n log n)**
Average Case	**O(n log n)**
Worst Case	**O(n²)**

Space Complexity:

- Average: **O(log n)** due to recursion
- Worst: **O(n)**

Advantages

- Fast on average
- In-place sorting
- Good cache performance
- Suitable for large datasets

Disadvantages

- Worst-case **O(n²)**
- Performance depends on pivot selection
- Recursive implementation uses stack space